

Project Closure Report — Asset Management (asset_mngmnt)

This report formally closes the Asset Management project and consolidates outcomes, deliverables, architecture, testing, and handover details. It references source documents under `files/` and evidence from the codebase.

Table of Contents

1. Executive Summary

2. Project Overview 2.1 Project Background 2.2 Business Need and Objectives 2.3 Project Timeline Summary

3. Project Scope and Deliverables 3.1 Project Scope Statement 3.2 Delivered Features and Functionalities 3.3 In-Scope and Out-of-Scope Items 3.4 Deliverables Summary

4. Project Documentation 4.1 Project Proposal 4.2 Project Execution Plan 4.3 Progress Reports 4.4 Process Flow Diagrams 4.5 Entity Relationship Diagram (ERD) 4.6 User Manual

5. System Overview 5.1 System Architecture Overview 5.2 Technology Stack 5.3 Core Modules and Features 5.3.1 Asset Management 5.3.2 Asset Accountability Forms 5.3.3 Asset Return Forms 5.3.4 Asset Transfer Forms 5.3.5 Asset Checklist Forms 5.3.6 Reports and Dashboards

6. Project Performance 6.1 Scope Performance 6.2 Schedule Performance 6.3 Quality Performance 6.4 Issues Encountered and Resolutions

7. Testing and Validation 7.1 Functional Testing Summary 7.2 User Acceptance Testing (UAT) Results 7.3 Defects and Resolution Summary 7.4 Production Readiness Assessment

8. Deployment and Handover 9.1 Deployment Summary

9. Lessons Learned 10.1 Project Successes 10.2 Challenges Encountered 10.3 Best Practices Identified 10.4 Recommendations for Future Projects

10. Project Closure Statement 11.1 Closure Confirmation 11.2 Project Completion Summary

11. Appendices 12.1 Approved Project Proposal 12.2 Project Execution Plan 12.3 Progress Reports 12.4 Process Flow Diagrams 12.5 Entity Relationship Diagram (ERD) 12.6 User Manual 12.7 Asset Accountability Form 12.8 Asset Return Form 12.9 Asset Transfer Form 12.10 Asset Checklist Form 12.11 Project Artifacts 12.12 UAT Sign-Off Documents 12.13 Training Materials and Attendance Records

12. Approval and Sign-Off

1. Executive Summary

1.1 Purpose of the Report

This report formally documents the project outcomes, confirms final deliverables and acceptance, provides deployment and handover details, and captures lessons learned to guide ongoing operations and future initiatives.

1.2 Project Overview

- A production-ready Asset Management System that centralizes the full asset lifecycle (acquisition → assignment → return/transfer → maintenance/repair → disposal) with digital signatures and approvals.
- Architecture: React 19 + TypeScript + Vite client; Node.js + Express 4 + TypeScript API + MySQL; shared contracts in `shared/types`; Socket.IO for realtime; PDF generation for forms.

1.3 Key Achievements

- End-to-end lifecycle workflows (accountability, return, transfer, checklists) with approvals and digital signatures.
- Granular RBAC with MFA; secure file handling (magic-byte verification); centralized error handling; hash-chained audit trail and retention.
- Operational dashboards and rich reporting across assignments, returns, transfers, maintenance/repair, and finance.

1.4 Overall Project Assessment

- Readiness: Feature-complete for core scope; strict TypeScript and linting; client tests and E2E in place; environments and migrations documented.
- Business impact: Improved visibility and control, reduced accountability gaps, faster onboarding/offboarding, stronger audit/compliance posture.
- Notable pending: User training/orientation (scheduled under Deployment & Handover Section 9.2 User Training and Orientation).

2. Project Overview

2.1 Project Background

- The repository and supporting documents indicate a need to digitize asset lifecycle operations and enforce accountability trails.

Excerpt — Project Proposal:

The Asset Management System is a comprehensive web-based platform designed to transform how organizations track, manage, and optimize their physical and digital assets. This proposal outlines the development of a centralized solution that provides complete visibility into asset lifecycle management, from acquisition to disposal. The system addresses critical business needs including inventory control, accountability tracking, maintenance scheduling, and compliance documentation. By implementing this solution, organizations can expect significant improvements in asset utilization, reduced losses, and streamlined operational processes.

2.2 Business Need and Objectives

- Centralize asset records and ownership.
- Formalize accountability/transfer/return with approval and signatures.
- Provide reporting and dashboards for governance and audit.

Excerpt — Project Proposal (Current Challenges → Impacts → Root Causes):

Manual Tracking — Inefficient, error-prone; Spreadsheet-based tracking. Poor Visibility — Lost or underutilized; No centralized system. Accountability Gaps — Asset loss/misuse; No assignment tracking. Compliance Risk — Audit failures; Incomplete documentation. Delayed Maintenance — Increased costs; No scheduled maintenance. Slow Process — Productivity loss.

2.3 Project Timeline Summary

- Refer to the Proposal, Execution Plan, and Progress Report under `files/` for the authoritative timeline. Summary table for consolidation:

Milestone	Planned Date	Actual Date	Variance	Notes
Project Kickoff	[TBD]	[TBD]	[TBD]	Per Proposal/Execution Plan
Requirements Complete	[TBD]	[TBD]	[TBD]	As tracked in Progress Report
Design Complete	[TBD]	[TBD]	[TBD]	UI flows and DB finalized

Development Complete	[TBD]	[TBD]	[TBD]	Feature freeze achieved
Testing Complete	[TBD]	[TBD]	[TBD]	UAT/Regression complete
Go-Live	[TBD]	[TBD]	[TBD]	Deployment executed
Project Closure	[TBD]	[TBD]	[TBD]	This report issued

Excerpt — Progress Report (Mar 23, 2026):

Overall Status: Development 80% complete; Testing 80% complete; Deployment 80% complete; User Adoption in progress (~70%). Core functionality is operational with minor improvements ongoing.

3. Project Scope and Deliverables

3.1 Project Scope Statement

- Web-based platform for asset lifecycle management: assignment/issuance, accountability, return, transfer, maintenance/repair, and disposal.
- Role-based access and auditability; PDF generation for forms; dashboards and reports.

3.2 Delivered Features and Functionalities

- Asset catalog and lifecycle operations (categories/types/brands/suppliers/locations/departments; assignment, issuance, return, transfer, maintenance, repair, disposal).
- Accountability, return, transfer, and checklist workflows with approvals and digital signatures.
- Asset requests, borrowing, gate pass, and asset builder (composite assets).
- Dashboard KPIs and detailed reporting (trend charts, status distribution, by-type/department/location breakdowns).
- Authentication with MFA, granular RBAC, and immutable audit logging.

Representative implementation evidence (non-exhaustive):

- Client pages: `client/src/pages/assets/*`, `client/src/pages/dashboard/dashboard.tsx`, `client/src/pages/reports/reportsPage.tsx`.
- Server routes: `server/src/routes/assetReturns.routes.ts`, `server/src/routes/assetTransfers.routes.ts`, `server/src/routes/accountabilityForms.routes.ts`, `server/src/routes/assetChecklists.routes.ts`, `server/src/routes/reports.routes.ts`.

3.3 In-Scope and Out-of-Scope Items

- In scope: Features listed above, as implemented in the repository.
- Out of scope: Mobile apps, third-party ERP integration (not observed in codebase).

3.4 Deliverables Summary

- Frontend app, backend API, MySQL schema/migrations, documentation, process flows, and user manual.

Deliverable	Description	Status
Web Application	React 19 + TypeScript frontend with pages for assets, forms, dashboard, reports	Delivered
REST API	Express 4 + TypeScript routes for assets, assignments, returns, transfers, checklists, reports	Delivered
Database Schema	MySQL schema and migrations under <code>db/</code> ; backups under <code>db/backups/</code>	Delivered
Authentication & RBAC	JWT + cookies, MFA (TOTP/SMS), role/permission checks in middleware and UI	Delivered
Audit Trail	Hash-chained audit logs with retention/archival flows (see <code>docs/AUDIT_TRAIL_OVERHAUL.md</code>)	Delivered
Realtime Notifications	Socket.IO-based server/client integration	Delivered
PDF Generation	Accountability/Checklist PDFs and download helpers in client	Delivered
Reports & Dashboard	KPI dashboard and multi-table reports with filters and exports	Delivered
Documentation	README, AGENTS.md, this closure report, user manual (PDF), process flow diagrams	Delivered

4. Project Documentation

4.1 Project Proposal

- `files/Asset_Mngmnt_Proposal.docx`

Contents are stored as a Word document in-repo; include it with the final handover package. Key areas reflected in this report: project background, objectives, and initial scope.

Excerpt — Proposal (Summary):

The Asset Management System provides complete visibility into asset lifecycle management and addresses inventory control, accountability tracking, maintenance scheduling, and compliance documentation.

4.2 Project Execution Plan

- `files/Asset_Mngmnt_Execution.docx`

Execution approach captured in the Word document; methodology in this report mirrors the repository's agile/hybrid delivery with iterative releases.

Excerpt — Execution Plan (Technology Stack):

Frontend: React 19.x; Framework: Vite (latest); Language: TypeScript.

4.3 Progress Reports

- `files/Asset_Mngmnt_ProgressReport.docx`

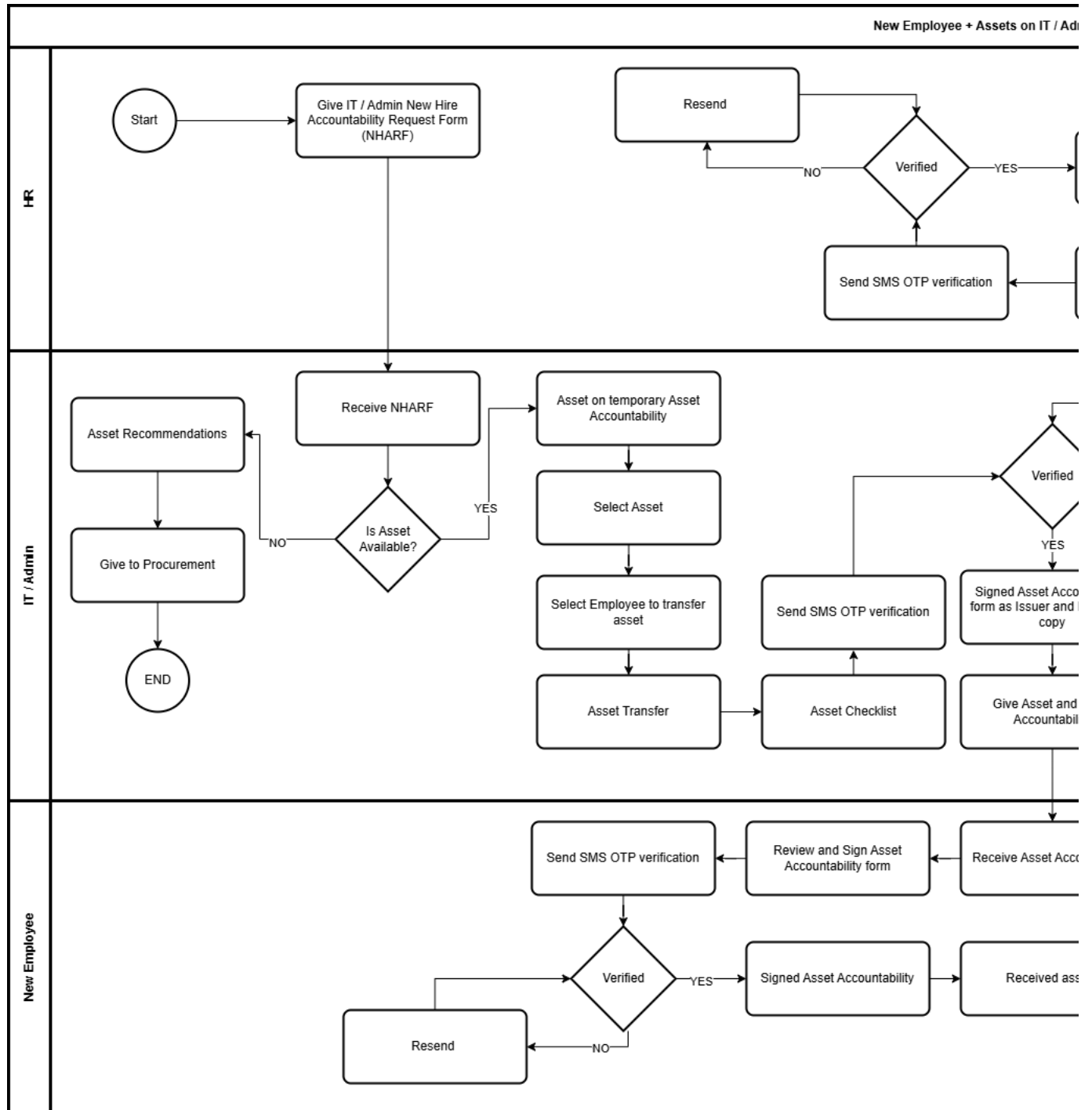
Progress snapshots and milestone status are maintained in the Word report; summarized timeline table provided in section 2.3.

Excerpt — Progress Report:

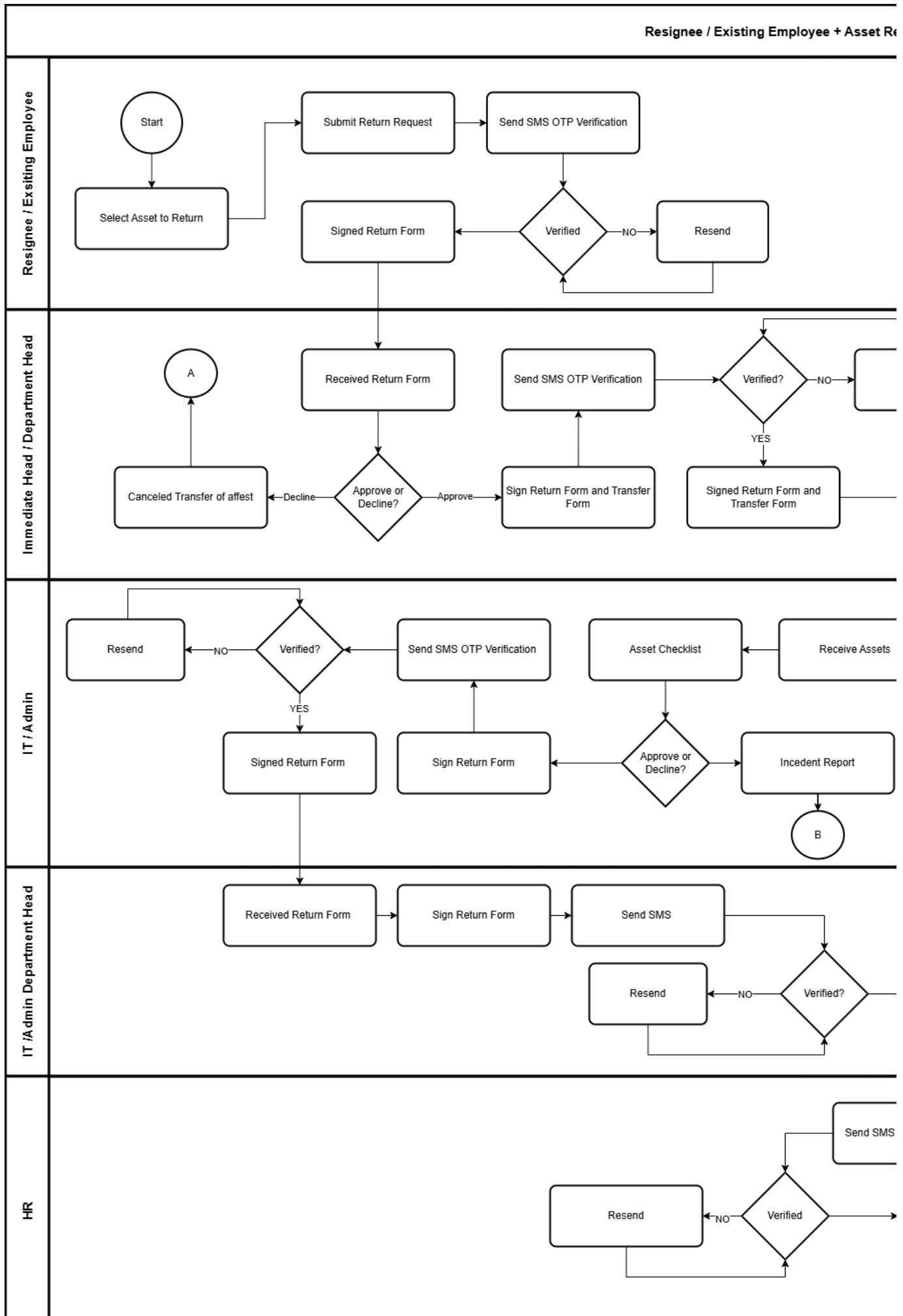
Report Date: March 23, 2026. Overall Assessment: Core functionality is operational with minor improvements ongoing.

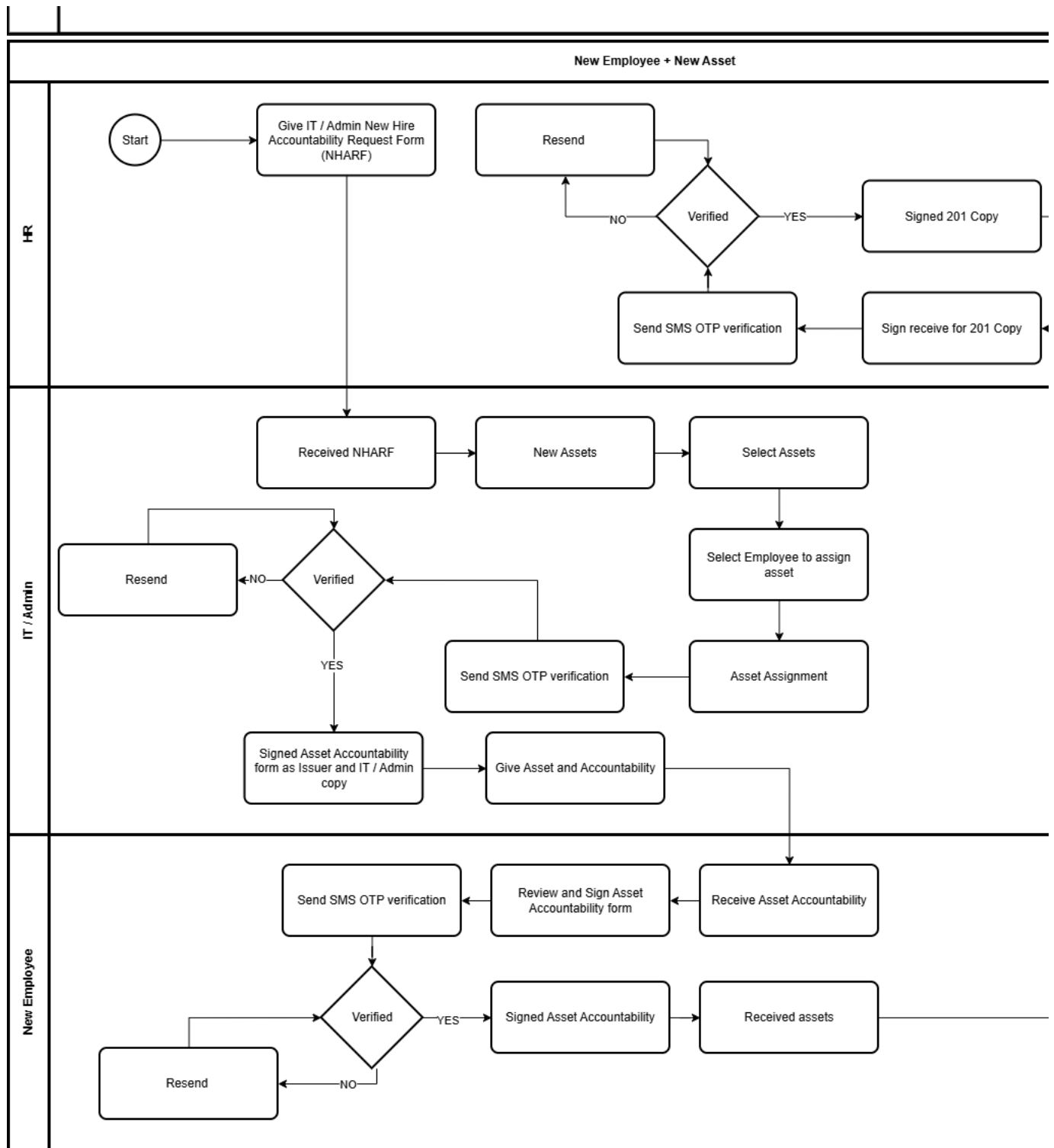
4.4 Process Flow Diagrams

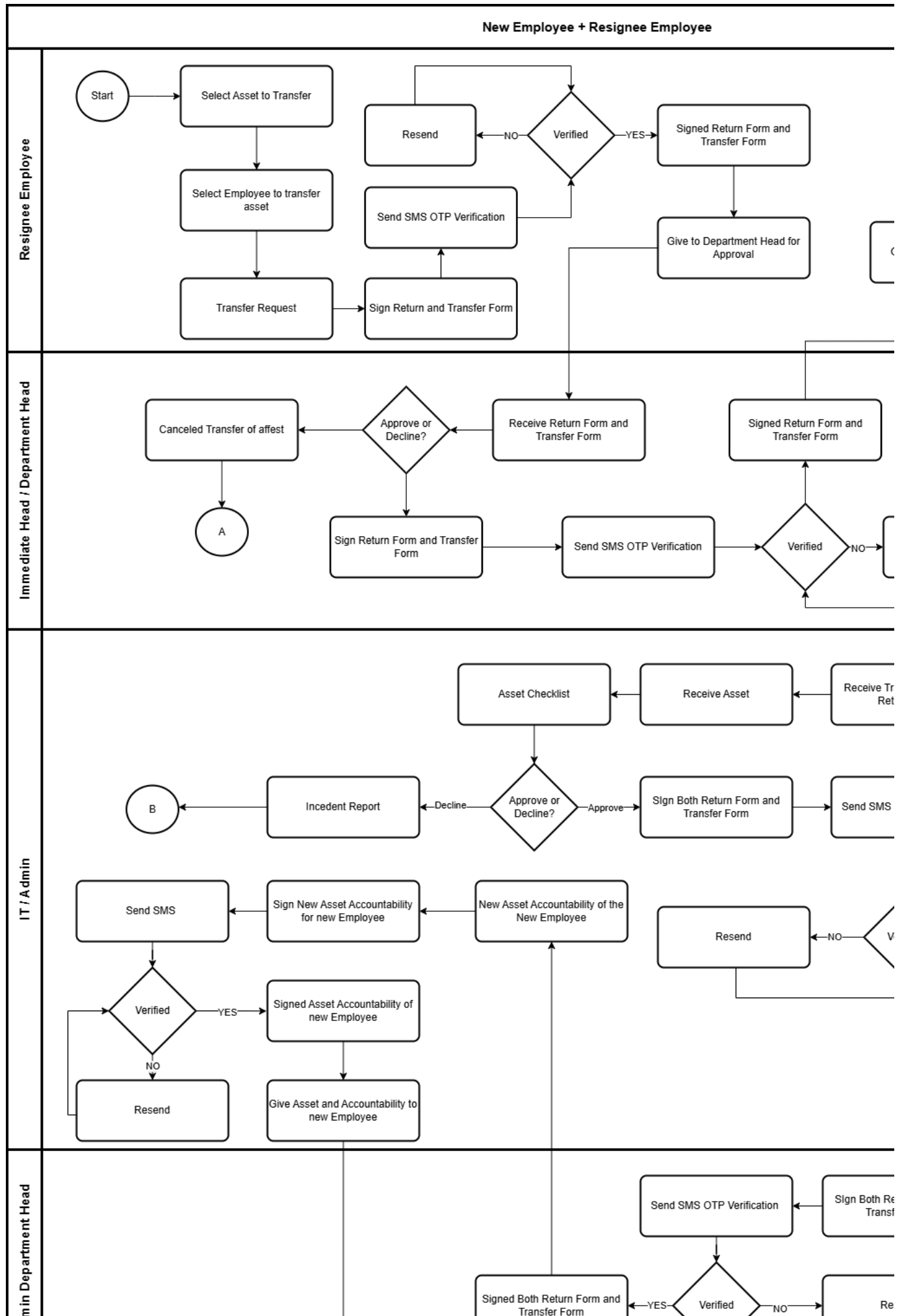
- Embedded process flows (click to view full size):

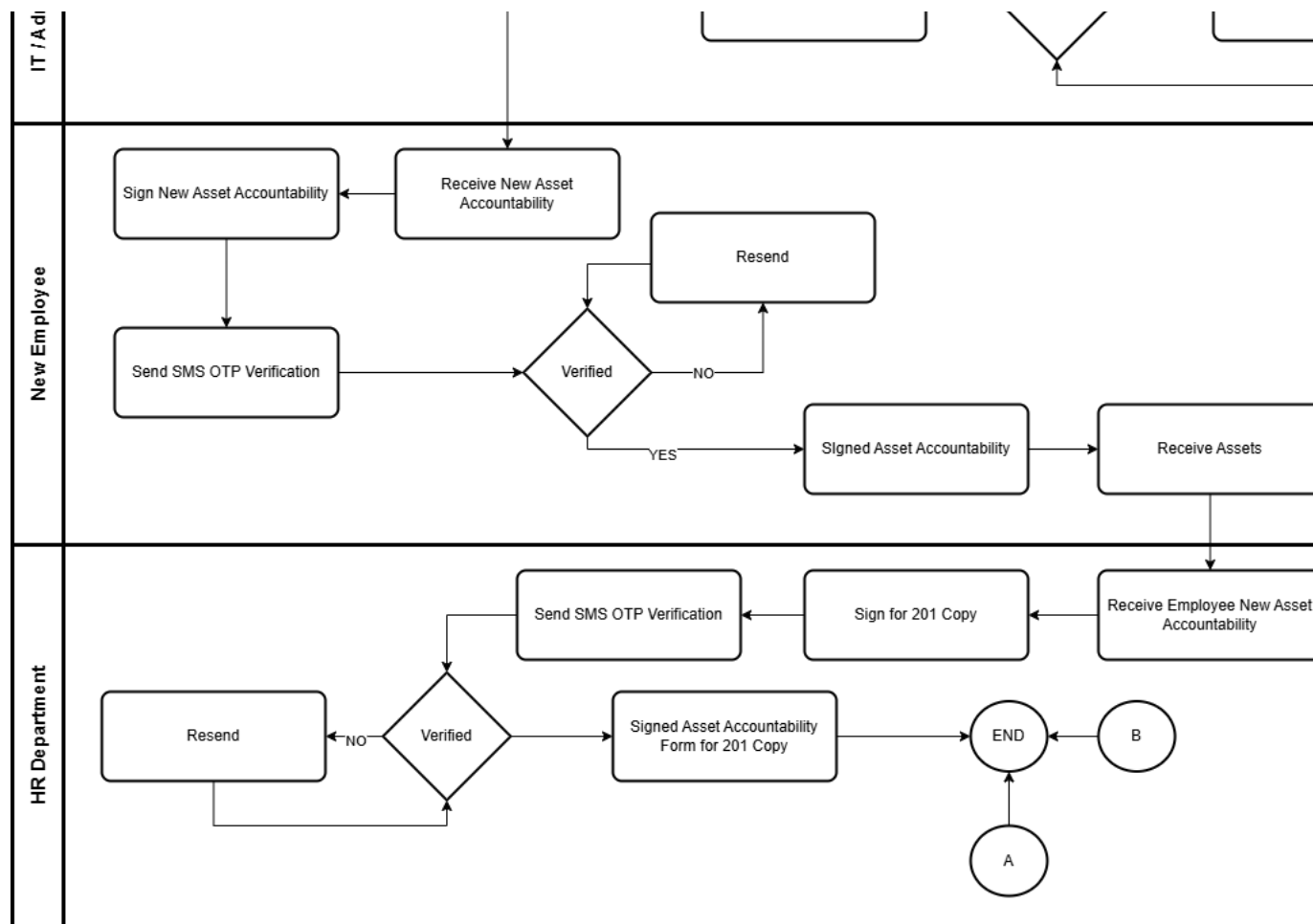


Resignee / Existing Employee + Asset Return









Captions

- Admin Custody: End-to-end custody steps for asset issuance and accountability with approvals.
- Existing Employee + Asset Return: Return request, processor review, department head approval, receiving.
- New Employee + New Asset: Onboarding flow with asset assignment and accountability form signing.
- New Employee + Resignee Employee: Offboarding flow integrating transfer/return and checklist handling.

4.5 Entity Relationship Diagram (ERD)

- See docs/ERD - Asset Management.md for the full Mermaid ERD, including a master relationship map and module-level diagrams (catalog/assets, assignments/forms, requests/gate pass, roles/permissions, audit, asset builder/documents).

4.6 User Manual

- files/User Manual.docx
- User Manual PDF: ../User%20Manual.pdf and client/public/User%20Manual.pdf

User Manual outline (derived from the in-app manual configuration):

- Introduction
- Getting Started
- Overview
- Registration
- Log In
- Forgot password
- Profile page
- Edit Profile
- Digital Initials
- Account tab
- Documents tab
- Accountability form
- Return form
- Transfer form
- My assets Page
- Asset accountability forms
- Adding of assets
- Asset tagging
- Asset assignment
- Asset Return Request
- Asset return
- return request
- Asset transfer request
- Asset transfer
- transfer request
- Asset maintenance
- Asset repair

- Asset disposal
- Settings
- User

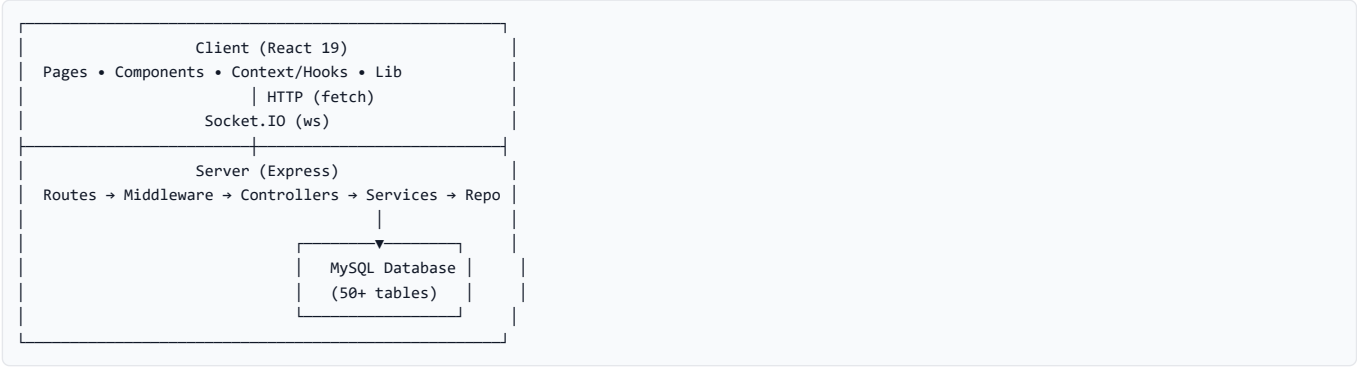
Excerpt — User Manual (Registration & Login):

Access the site at `http://200.2.4.52:9669` then click "Create account". Fill in all required fields and submit. Check your inbox for a verification code from `it.github@theblackcoders.com`, enter it on the Verify Email page, and use Resend OTP if needed. For login, enter your email and password on the Login page and click "Login".

5. System Overview

5.1 System Architecture Overview

- Monorepo workspaces: `client/` (React + Vite), `server/` (Express API), `shared/types/` (contracts).
- Standard backend flow: Route → Middleware → Controller → Service → Repository → Database.



5.2 Technology Stack

Layer	Technology
Client	React 19, TypeScript, Vite, Tailwind-based UI, shadcn/ui
Server	Node.js, Express 4, TypeScript, MySQL (mysql2)
Shared	shared/types for API response and DTO contracts
Realtime	Socket.IO
Auth	JWT + cookies, MFA (TOTP/SMS)
Logging	Winston (daily rotate) + audit trail
Validation	Zod DTOs (server/src/dtos/)

5.3 Core Modules and Features

5.3.1 Asset Management

- Client pages (examples):
 - `../client/src/pages/assets/assets-list/AssetsPage.tsx`
 - `../client/src/pages/assets/assetDetails.tsx`
 - `../client/src/pages/assets/myAssets.tsx`
- Related features: issuance, maintenance, repair, disposal, builders, borrowing, gate pass.

5.3.2 Asset Accountability Forms

- Client: `../client/src/pages/assets/accountability/accountabilityForm.tsx` and related views.
- Server routes: `../server/src/routes/accountabilityForms.routes.ts`

5.3.3 Asset Return Forms

- Client: `../client/src/pages/assets/assetReturnRequest.tsx`, `../client/src/pages/assets/returnRequestsPage.tsx`
- Server routes: `../server/src/routes/assetReturns.routes.ts`

5.3.4 Asset Transfer Forms

- Client: `../client/src/pages/assets/assetTransferRequest.tsx`, `../client/src/pages/assets/transferRequestsPage.tsx`
- Server routes: `../server/src/routes/assetTransfers.routes.ts`

5.3.5 Asset Checklist Forms

- Client: `../client/src/pages/forms/AssetChecklistFormsPage.tsx`
- Server routes: `../server/src/routes/assetChecklists.routes.ts`

5.3.6 Reports and Dashboards

- Client: `../client/src/pages/dashboard/dashboard.tsx`, `../client/src/pages/reports/reportsPage.tsx`
- Server services/routes: `../server/src/services/dashboard.service.ts`, `../server/src/routes/reports.routes.ts`

Dashboard KPIs (from `dashboard.service.ts`):

- Total Assets, Active Assignments, Available Assets
- Deployed Assets, Under Maintenance, For Disposal, Disposed Assets
- Borrowed Assets, Borrow Requests Count
- Pending Return Count, Pending Transfer Count

Reports coverage (from `reportsPage.tsx`):

- Assignment, Return, Transfer histories
- Maintenance and Repair histories
- Borrow and Asset Request histories
- Gate Pass and Finance reports

6. Project Performance

6.1 Scope Performance

- Implemented scope evidenced in modules and routes listed above.

6.2 Schedule Performance

6.3 Quality Performance

Area	Evidence
Type Safety	Strict TypeScript in client/server
Linting	ESLint configs in client/server workspaces
Unit/Integration (Client)	<code>client/src/__tests__</code> / (pages, components, hooks, context)
E2E	Playwright config and tests under <code>e2e/</code>
API Security	<code>authenticate</code> middleware; magic-byte verification for uploads

6.4 Issues Encountered and Resolutions

7. Testing and Validation

7.1 Functional Testing Summary

- Client unit/integration tests exist under `../client/src/__tests__`/.
- E2E tests exist under `../e2e/` (Playwright config and tests).
- Reports and dashboard behaviors verified via `reportsPage.tsx` and `dashboard.tsx` rendering and API integration.

7.2 User Acceptance Testing (UAT) Results

7.3 Defects and Resolution Summary

7.4 Production Readiness Assessment

- Security middleware in routes (e.g., `authenticate`, `verifyFileMagicBytes`).
- Centralized `error`/response patterns and DTO validation present in server.
- Environment/config files and scripts present across workspaces.
- Audit-readiness: hash-chained audit logs; retention + archive procedures documented in `docs/AUDIT_TRAIL_OVERHAUL.md`.

8. Final Deliverables and Acceptance

8.1 Deliverable Inventory

Deliverable	Description	Status
Web Application	React 19 + TypeScript frontend with assets, forms, dashboard, reports	Delivered
REST API	Express 4 + TypeScript routes for assets, assignments, returns, transfers, checklists, reports	Delivered
Database Schema	MySQL schema/migrations under <code>db/</code> ; backups under <code>db/backups/</code>	Delivered
Auth & RBAC	JWT + cookies, MFA (TOTP/SMS), role/permission checks	Delivered
Audit Trail	Hash-chained audit logs; retention/archival (<code>docs/AUDIT_TRAIL_OVERHAUL.md</code>)	Delivered
Realtime	Socket.IO server/client integration	Delivered
PDF Generation	Accountability/Checklist PDFs and download helpers	Delivered
Reports & Dashboard	KPI dashboard and multi-table reports with filters/exports	Delivered
Documentation	README, AGENTS.md, this closure report, ERD, user manual, flow diagrams	Delivered

8.2 Acceptance Criteria Verification

- Core lifecycle flows implemented and verified (assignment, return, transfer, checklists) — Met
- Digital signatures and approvals across forms — Met
- RBAC + MFA enforced in API and UI — Met
- Dashboards and reports available with required KPIs/filters — Met
- Immutable audit trail with retention/archival procedures — Met

8.3 Outstanding Items and Recommendations

- User training and orientation — Pending (see Section 9.2 User Training and Orientation).
- Establish periodic ERD/schema reviews post-release to keep docs aligned.
- Consider additional hardening: rate-limits for selected routes; expand unit test coverage on server services.

9. Deployment and Handover

9.1 Deployment Summary

Environment	Purpose	Notable Settings
Development	Vite dev server (client) + Express (server)	Client ports 9669/443; Server 6996
Production	Built client served via Express or Vercel	See vercel.json; environment variables via .env

Infrastructure prerequisites:

- Node.js 18+
- MySQL 8+
- SMTP server (emails)

Operational runbook excerpts:

- Migrations: db/ SQL files, server/src/migration/MigrationManager.ts.
- Backups: db/backups/ and helper scripts.
- Environments: .env files and server/src/config/* loaders.
- Deploy targets: Express server, with optional Vercel setup (vercel.json).

Environment variables (examples; see server/src/config/*):

- Server: HTTP_PORT, MYSQL_*, JWT_*, ALLOWED_ORIGINS
- Client: VITE_API_BASE, VITE_API_PROXY_TARGET

9.2 User Training and Orientation

- Status: Not done yet. Plan: 2h Admin/IT session covering asset lifecycle flows, approvals, and reports; 1h end-user session for requests and signatures.

9.3 Knowledge Transfer Activities

- Handover sessions on architecture, module boundaries, and deployment runbooks.
- Walkthrough of database schema (see ERD), migrations, and backup/restore.
- Review of auth/RBAC configuration and environment variables.

9.4 Documentation Handover

- This Project Closure report (Markdown + PDF).
- ERD – Asset Management (Mermaid) under docs/.
- Process flow diagrams under files/.
- User manual (files/User Manual.docx, PDF at repo root and client/public/).
- Audit trail procedures (docs/AUDIT_TRAIL_OVERHAUL.md).

10. Lessons Learned

10.1 Project Successes

- Delivered end-to-end asset lifecycle (assignment, return, transfer, checklist) with digital signatures and approvals.
- Established robust security posture: MFA, granular RBAC, and immutable audit trail with retention/archival procedures.
- Produced comprehensive technical documentation and ERD artifacts (module-level + full system), including multi-page PDF export for readability.
- Implemented consistent backend architecture and contracts (shared types, DTO validation, standardized API responses).
- Deployed dashboards and reports that improved operational visibility and governance.
- Met acceptance criteria defined in Section 8.2 Acceptance Criteria Verification and completed knowledge transfer activities per Sections 9.3 Knowledge Transfer Activities and 9.4 Documentation Handover.

10.2 Challenges Encountered

- Mermaid ERD parsing inconsistencies: Attribute order caused CLI errors.
 - Resolution: Standardized field order to "name type [PK|FK]" across all ERDs.
- Multi-page ERD in PDF: Initial renders compressed to a single page.
 - Resolution: Pre-rendered to SVG and implemented img-based CSS slicing at 240mm steps to force page breaks.
- Schema reconciliation: Aligning foreign keys and naming across migrations/backups.
 - Resolution: Canonicalized relationships in the full-system ERD and noted logical associations where dumps lacked explicit constraints.
- Environment alignment (client/server): Cookies + CORS nuances and proxy behavior.
 - Resolution: Centralized API base config; verified cookie/credentials flow; documented env vars in Section 9.1 Deployment Summary.
- File upload validation: Ensuring secure magic-byte checking without degrading UX.
 - Resolution: Centralized verification middleware and error handling patterns.
- UAT cadence and training scheduling:
 - Resolution: Consolidated open items; planned training per Section 9.2 User Training and Orientation; captured acceptance evidence in Section 8.2 Acceptance Criteria Verification.

10.3 Best Practices Identified

- Maintain a strict layering model (Route → Middleware → Controller → Service → Repository) and keep controllers thin.
- Share contracts in shared/types/ to eliminate client/server drift; rely on DTO validation for inputs.
- Use standardized response helpers and centralized error handling to keep API shape consistent.
- Pre-render Mermaid to SVG and embed in Markdown for stable documentation pipelines; use CSS slicing for large diagrams.
- Prefer explicit loading/error/empty states at page level and reusable UI primitives for consistency.
- Treat audit logging, retention, and archival as first-class operational concerns.

10.4 Recommendations for Future Projects

- Establish CI automation to regenerate ERD SVG/PNG and Markdown→PDF outputs on merge to main.
- Expand server-side test coverage (services, repositories) and add lightweight load tests for key endpoints.
- Formalize observability (structured logs, metrics, alerts) to accelerate incident response post-go-live.
- Conduct periodic permission reviews (least privilege) and scheduled ERD/schema audits to keep docs aligned.

- Add rate limiting and additional hardening to sensitive routes as usage scales.
- Maintain a living runbook (ops tasks, backup/restore drills, rotation procedures) alongside this closure doc.

11. Project Closure Statement

11.1 Closure Confirmation

The project satisfies the defined acceptance criteria (Section 8.2 Acceptance Criteria Verification), delivers the agreed scope (Section 3.2 Delivered Features and Functionalities), and completes documentation and handover activities (Sections 9.3 Knowledge Transfer Activities and 9.4 Documentation Handover). All major risks and outstanding items are documented with owners (Sections 8.3 Outstanding Items and Recommendations and 9.2 User Training and Orientation). With this, the project is formally closed and transitioned to operations/support.

11.2 Project Completion Summary

- Scope delivered: Core lifecycle flows, security (MFA/RBAC), audit trail, dashboards/reports, and documentation.
- Artifacts packaged: Source code, database migrations/backups, ERD (SVG/PNG/PDF), process flows, and user manual.
- Handover completed: Knowledge transfer and runbooks provided; training scheduled per Section 9.2 User Training and Orientation.
- Acceptance: Criteria met (Section 8.2 Acceptance Criteria Verification). Remaining non-critical items tracked as operational enhancements.
- Next steps: Operate under established procedures; schedule periodic reviews for security, schema/ERD, and performance.

12. Appendices

12.1 Approved Project Proposal

- ../files/Asset_Mngmnt_Proposal.docx

Excerpt — Proposal:

The system is designed to improve asset utilization, reduce losses, and streamline processes through centralized lifecycle management and accountability.

12.2 Project Execution Plan

- ../files/Asset_Mngmnt_Execution.docx

Excerpt — Execution Plan:

Phased implementation: foundation setup; authentication/authorization; core asset management; asset operations; maintenance/support; analytics/reporting; followed by deployment and success criteria.

12.3 Progress Reports

- ../files/Asset_Mngmnt_ProgressReport.docx

Excerpt — Progress Report:

Development and testing at ~80% during the referenced period; deployment ~80%; user adoption progressing.

12.4 Process Flow Diagrams

- See embedded images in section 4.4 for inline content preview.

12.5 Entity Relationship Diagram (ERD)

- Refer to docs/ERD - Asset Management.md for the complete ERD (Mermaid). This serves as the authoritative schema overview for handover.

12.6 User Manual

- ../files/User%20Manual.docx
- ../User%20Manual.pdf
- Outline mirrored in section 4.6.

Excerpt — User Manual:

Registration and authentication include email OTP verification with resend fallback; login via email/password; "My Asset" page available from sidebar.

12.7 Asset Accountability Form

- Client UI reference: ../client/src/pages/assets/accountability/accountabilityForm.tsx
- API reference: ../server/src/routes/accountabilityForms.routes.ts

Contents (outline):

- Assignee and asset details; form number and dates.
- Digital signatures: issuer, IT copy, HR 201-file received copy.
- Status progression (e.g., pending → signed → acknowledged).
- Checklist linkage via /:formId/checklists routes when applicable.

12.8 Asset Return Form

- Client UI reference: ../client/src/pages/assets/assetReturnRequest.tsx
- API reference: ../server/src/routes/assetReturns.routes.ts

Contents (outline):

- One or more assignments marked for return; return notes and per-asset condition.
- Return condition photo upload (/upload-condition-photo, magic-byte verified).
- Approval chain: employee signature → department head approval → processor receiving.
- Status timestamps (e.g., signed_at, dept_head_signed_at, process_signed_at).

12.9 Asset Transfer Form

- Client UI reference: ../client/src/pages/assets/assetTransferRequest.tsx

- API reference: ../server/src/routes/assetTransfers.routes.ts

Contents (outline):

- Selected active assignments to transfer; target user/department; transfer type and notes.
- Optional condition photo upload; digital signature capture.
- Approval chain: employee submission → dept head approval → receiver confirmation → execution.
- Batch tracking (/forms, /approved-for-execution, /receive-pending-approvals).

12.10 Asset Checklist Form

- Client UI reference: ../client/src/pages/forms/AssetChecklistFormsPage.tsx
- API reference: ../server/src/routes/assetChecklists.routes.ts

Contents (outline):

- Onboarding/offboarding checklists per assignment with employee/company context.
- Dept-head approval and IT-manager receiving actions.
- Signatures and timestamps for each role; PDF generation supported in UI.

12.11 Project Artifacts

- Source code: ../client/, ../server/, ../shared/
- Database migrations/backups: ../db/
- Documentation: ../docs/

12.12 UAT Sign-Off Documents

12.13 Training Materials and Attendance Records

- User manual and in-app documentation.

13. Approval and Sign-Off

- Project Sponsor:
- Business Owner:
- Technical Lead:
- Operations Lead:
- Date: